

```
# Aggregation:
# Part of Object that is if do not have, the object works.
# AC: if AC is not in Car, The car will work
# Car is different class, AC is different class
```

```
# Composition
# Part of object that if do not have, the object will ont work
# Engine : The car will not work without Engine.
# Car is different class, Engine is different class
```

```
# Association
# Two classes (completely different) object indirect relationship
# The third class will include both these classes.
```

```
# Method Overriding
# same function name use for different object differently
```

```
class Car:
    vehType = "Car" # Class variable

    def __init__(self, wheels, make, model, color):
        self.wheels = wheels # Instance Attribute
        self.make = make # Instance Attribute
        self.model = model # Instance Attribute
        self.color = color # Instance Attribute

    def move(self, txt="The"):
        print(f"{txt} {self.color} {self.model} {self.vehType} is moving.")
```

```
mycar = Car(4, "Toyota", "Corolla", "Red") # "mycar" is an objet
print(f"My {mycar.vehType} is a {mycar.color} {mycar.make} {mycar.model}.")
```

```
My Car is a Red Toyota Corolla.
```

```
your_car = Car(4, "Honda", "Civic", "Blue")
print(f"My {your_car.vehType} is a {your_car.color} {your_car.make} {your_car.model}.")
```

```
My Car is a Blue Honda Civic.
```

```
mycar.move("My")
your_car.move("Your")
```

```
My Red Corolla Car is moving.
Your Blue Civic Car is moving.
```

```
info = f"My {mycar.wheels} wheeler, {mycar.color} {mycar.make} {mycar.model}"
print(info, mycar.vehType)
```

```
My 4 wheeler, Red Toyota Corolla Car
```

```
mycar.move(info)
```

```
My 4 wheeler, Red Toyota Corolla Red Corolla Car is moving.
```

```
## Encapsulation ##
```

```
class Car:
    __vehType = "Car"

    def __init__(self, wheels, make, model, color):
        self.__wheels = wheels # Private Instance
        self.__make = make # Private Instance
        self.__model = model # Private Instance
        self.color = color # Public Instance
```

```
def move(self, txt="The"):
    print(f"{txt} {self.__vehType} is moving.")
```

```
my_car = Car(4, "Toyota", "Camry", "Blue")
print(f"The color of my car is {mycar.color}.")
```

The color of my car is Red.

```
my_car.move("My")
```

My Car is moving.

▼ Private Variable

```
print(f"My {my_car.wheels} wheeler, {my_car.color} {my_car.make} {my_car.model}")
```

```
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_10931/4258669758.py in <cell line: 0>()
----> 1 print(f"My {my_car.wheels} wheeler, {my_car.color} {my_car.make} {my_car.model}")

AttributeError: 'Car' object has no attribute 'wheels'
```

```
class Car:
    __vehType = "Car"

    def __init__(self, wheels, make, model, color):
        self.__wheels = wheels    # Private Instance
        self.__make = make        # Private Instance
        self.__model = model      # Private Instance
        self.color = color        # Public Instance
# getter
    def get_vehType(self):
        return self.__vehType
    def get_wheels(self):
        return self.__wheels
    def move(self, txt="The"):
        print(f"{txt} {self.__vehType} is moving.")
```

```
my_car = Car(4, "Toyota", "Camry", "Blue")
```

```
print(f"It is {my_car.get_wheels()} wheeler {my_car.get_vehType()}.")
```

It is 4 wheeler Car.

```
print(f"My {my_car.get_vehType()} is moving.")
```

My Car is moving.

```
class Car:
    __vehType = "Car"

    def __init__(self, wheels, make, model, color):
        self.__wheels = wheels    # Private Instance
        self.__make = make        # Private Instance
        self.__model = model      # Private Instance
        self.color = color        # Public Instance
# getter
    def get_vehType(self):
        return self.__vehType
    def get_wheels(self):
        return self.__wheels
    def get_make(self):
        return self.__make
    def get_model(self):
        return self.__model
    def move(self, txt="The"):
        print(f"{txt} {self.__vehType} is moving.")
```

```
mycar = Car(4, "Toyota", "Camry", "Blue")
```

```
print(f"My {my_car.get_vehType()} is moving.")
print(f"My car maker is {mycar.get_make()} and my model is {mycar.get_model()}")
```

```
My Car is moving.
My car maker is Toyota and my model is Camry.
```

```
class Car:
    __vehType = "Car"

    def __init__(self, wheels, make, model, color):
        self.__wheels = wheels    # Private Instance
        self.__make = make        # Private Instance
        self.__model = model      # Private Instance
        self.color = color        # Public Instance
# getter
    def get_vehType(self):
        return self.__vehType
    def get_wheels(self):
        return self.__wheels
    def get_info(self):
        return f"Wheels : {self.__wheels}, Make : {self.__make}, Model : {self.__model}, Color : {self.color}"
    def move(self, txt="The"):
        print(f"{txt} {self.__vehType} is moving.")
```

```
mycar = Car(4, "Toyota", "Camry", "Blue")
```

```
print(f"It is {mycar.get_wheels()} wheeler {mycar.get_vehType()}")
```

```
It is 4 wheeler Car.
```

```
info = mycar.get_info()
print(info)
```

```
Wheels : 4, Make : Toyota, Model : Camry, Color : Blue
```

✓ INHERITANCE

```
class Vehicle:
    __vehType = "Vehicle"

    def __init__(self, wheels, make, model, color):
        self.__wheels = wheels    # Private Instance
        self.__make = make        # Private Instance
        self.__model = model      # Private Instance
        self.color = color        # Public Instance

    def set_vehType(self, vehType): # setter are usually "procedures"
        self.__vehType = vehType

    def get_vehType(self):
        return self.__vehType

    def get_wheels(self):
        return self.__wheels
    def get_info(self):
        return f"Wheels : {self.__wheels}, Make : {self.__make}, Model : {self.__model}, Color : {self.color}"
    def move(self, txt="The"):
        print(f"{txt} {self.__vehType} is moving.")
```

✓ Derived class

```
class Car(Vehicle):
    def __init__(self, wheels, make, model, color):
        super().__init__(wheels, make, model, color)
        self.set_vehType("Car")

class Motorcycle(Vehicle):
```

```
def __init__(self, wheels, make, model, color):
    super().__init__(wheels, make, model, color)
    self.set_vehType("Motorcycle")
```

```
Mycar = Car(4, "Toyota", "Camry", "Blue")
```

```
print(f"It is {Mycar.get_wheels()} wheeler {Mycar.get_vehType()}.")
```

```
It is 4 wheeler Car.
```

```
Mycar.get_vehType()
```

```
'Car'
```

```
Mybike = Motorcycle(2, "Yamaha", "MT-07", "Red")
```

```
print(f"It is {Mybike.get_wheels()} wheeler {Mybike.get_vehType()}.")
```

```
It is 2 wheeler Motorcycle.
```

```
Mybike.get_vehType()
```

```
'Motorcycle'
```

```
info = Mybike.get_info()
print(info)
```

```
Wheels : 2, Make : Yamaha, Model : MT-07, Color : Red
```

```
Mycar.move()
```

```
The Car is moving.
```

```
Mybike.move()
```

```
The Motorcycle is moving.
```

Method Overriding

```
class Vehicle:
    __vehType = "Vehicle"

    def __init__(self, wheels, make, model, color):
        self.__wheels = wheels    # Private Instance
        self.__make = make        # Private Instance
        self.__model = model      # Private Instance
        self.color = color        # Public Instance

    def set_vehType(self, vehType): # setter are usually "procedures"
        self.__vehType = vehType

    def get_vehType(self):
        return self.__vehType

    def get_wheels(self):
        return self.__wheels

    def get_info(self):
        return f"Wheels : {self.__wheels}, Make : {self.__make}, Model : {self.__model}, Color : {self.color}"

    def move(self, txt="The"):
        print(f"{txt} {self.__vehType} is moving.")
```

```
class Car(Vehicle):
    def __init__(self, wheels, make, model, color):
        super().__init__(wheels, make, model, color)
        self.set_vehType("Car")

    def move(self, txt="The"):
```

```
print(f"{txt} {self.get_vehType()} is being drivien.")

class Motorcycle(Vehicle):
    def __init__(self, wheels, make, model, color):
        super().__init__(wheels, make, model, color)
        self.set_vehType("Motorcycle")
    def move(self, txt="The"):
        print(f"{txt} {self.get_vehType()} is being ridden.")
```

```
mycar = Car(4, "Toyota", "Camry", "Blue")
mybike = Motorcycle(2, "Yamaha", "MT-07", "Red")
```

```
mycar.move("My")
```

My Car is being drivien.

```
mybike.move("Your")
```

Your Motorcycle is being ridden.

```
info = mycar.get_info()
print(info)
```

Wheels : 4, Make : Toyota, Model : Camry, Color : Blue

```
info = mybike.get_info()
print(mybike.get_info())
```

Wheels : 2, Make : Yamaha, Model : MT-07, Color : Red

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.